

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



HỆ CƠ SỞ DỮ LIỆU (CO3093)

Bài tập lớn

PHÁT TRIỂN ỨNG DỤNG MẠNG CHIA SẺ FILE PEER-TO-PEER

Giảng viên hướng dẫn: TS. Nguyễn Lê Duy Lai
Sinh viên: Phạm Ngọc Long - 2211894.
Võ Phương Minh Nhật - 2212413.
Nguyễn Quang Huy - 2211235.

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 10 NĂM 2024



Mục lục

1	Giới thiệu chung	2
1.1	Tổng quan dự án	2
1.2	Mục tiêu dự án	2
2	Cơ sở lý thuyết	2
2.1	Mạng peer-to-peer	2
2.1.1	Mạng Peer-to-peer là gì?	2
2.2	BitTorrent	3
2.2.1	BitTorrent là gì?	3
2.3	Các thành phần cơ bản	3
3	Các chức năng chính của ứng dụng	3
3.1	Tracker Server	3
3.2	Client	4
3.3	Giao thức giao tiếp của ứng dụng	6
3.3.1	Server và client	6
3.3.2	Client và client	7
4	Hiện thực ứng dụng	8
4.1	Diagram	8
4.1.1	Use-case diagram	8
4.1.2	Class diagram	8
4.2	Cấu trúc metainfo file	10
4.3	Hiện thực downloading	10
4.3.1	Thiết lập kết nối	11
4.3.2	Chiến lược tải file Rarest-First	11
4.3.3	Tải đồng thời nhiều file	11
4.4	Hiện thực uploading	12
4.5	Hiện thực các chức năng khác	12
5	Kết quả hiện thực	12
5.1	Chức năng Connect	12
5.2	Chức năng Create	13
5.3	Chức năng Publish	14
5.4	Chức năng Search	15
5.5	Chức năng Get_torrent	15
5.6	Chức năng My_torrent	16
5.7	Chức năng Download	16
5.8	Chức năng upload	19
6	Đánh giá hiệu suất	20
7	Nhiệm vụ và đóng góp của các thành viên	21
8	Những điều đạt được và hướng mở rộng	21
	Tài liệu tham khảo	22

1 Giới thiệu chung

1.1 Tổng quan dự án

Dự án này hiện thực một ứng dụng chia sẻ file theo cơ chế Peer-to-Peer (P2P), trong đó mỗi thiết bị trong mạng vừa đóng vai trò là máy chủ vừa là máy khách, cùng nhau chia sẻ tài nguyên và thực hiện tác vụ mà không cần đến một trung tâm điều khiển duy nhất. Thông qua mạng P2P, các thiết bị có thể trao đổi dữ liệu trực tiếp với nhau, tạo nên một hệ thống mạng linh hoạt và hiệu quả.

Trong ứng dụng này, một máy chủ tập trung, gọi là tracker, sẽ theo dõi các peer đang kết nối và lưu trữ mảnh nào của một file. Mỗi peer sẽ thông báo với tracker về những file hoặc mảnh file nào đang được lưu trữ cục bộ mà không cần gửi dữ liệu thực tế của các file này lên tracker. Khi một peer cần tải về một file chưa có trong kho lưu trữ, nó có thể gửi yêu cầu đến tracker để tìm các peer khác đang chia sẻ file đó. Ngoài ra, ứng dụng hỗ trợ tải xuống nhiều file từ cùng một peer tại cùng một thời điểm thông qua cơ chế đa luồng (multithreading), giúp tối ưu hóa tốc độ truyền tải và cải thiện trải nghiệm người dùng.

1.2 Mục tiêu dự án

Ứng dụng chia sẻ file dựa trên cơ chế Peer-to-Peer cho phép các máy gia nhập vào mạng chia sẻ file trực tiếp với nhau mà không cần thông qua server (chỉ cần tracker theo dõi thông tin các peer). Điều này làm cho hệ thống ít bị ảnh hưởng bởi single point of failure (lỗi xảy ra tại một điểm trọng yếu dẫn đến lỗi cho toàn bộ hệ thống - trong trường hợp cơ chế client-server thì server là point of failure). Việc chia sẻ file Peer-to-Peer còn làm tăng tốc độ tải về khi một file có thể được tải từ nhiều peer cùng một lúc. Không những thế, ứng dụng này còn đảm bảo về tài nguyên lưu trữ dữ liệu khi lượng dữ liệu cần cung cấp và số lượng peer tham gia vào mạng ngày càng tăng.

2 Cơ sở lý thuyết

2.1 Mạng peer-to-peer

2.1.1 Mạng Peer-to-peer là gì?

Kiến trúc peer-to-peer (P2P) là một mô hình mạng mà trong đó các máy tính hay peers (điểm nút) có vai trò tương đương và có thể giao tiếp trực tiếp với nhau mà không cần máy chủ trung tâm để điều phối. Trong mô hình này, mỗi peer có thể vừa là client (nhận dữ liệu) vừa là server (chia sẻ dữ liệu), giúp loại bỏ sự phụ thuộc vào một máy chủ duy nhất và làm cho hệ thống trở nên phân tán và linh hoạt hơn.

Đặc điểm chính của kiến trúc P2P:

- Không có máy chủ trung tâm, các điểm nút tự quản lý và giao tiếp với nhau.
- Các peer có thể chia sẻ tài nguyên như băng thông, dung lượng lưu trữ và công suất xử lý với nhau.
- Nếu một peer gặp sự cố, toàn bộ hệ thống vẫn có thể hoạt động, vì các điểm nút khác có thể tiếp tục giao tiếp. Điều này có tác dụng phân tán rủi ro, giảm khả năng hư hỏng của toàn bộ hệ thống.

2.2 BitTorrent

2.2.1 BitTorrent là gì?

BitTorrent là một giao thức phân phối file P2P phổ biến. Trong BitTorrent, tập hợp tất cả các peer tham gia vào việc phân phối một file cụ thể được gọi là một torrent. Các peer trong một torrent tải xuống các piece (mảnh) của file có kích thước bằng nhau (thường là 256 KB) từ các peer khác. Khi một peer mới tham gia vào một torrent, nó không lưu trữ piece nào của file và dần tích lũy nhiều piece hơn theo thời gian. Trong khi tải xuống các piece, nó cũng tải lên các piece cho các peer khác. Khi một peer đã tải xong toàn bộ file, nó có thể (dùng chiến thuật greedy - tham lam) rời khỏi torrent hoặc (dùng chiến thuật altruistic - vị tha) ở lại torrent và tiếp tục tải lên các piece cho các peer khác.

2.3 Các thành phần cơ bản

- Magnet text: là một đoạn văn bản đơn giản chứa tất cả các thông tin cần thiết cho tệp metainfo; yêu cầu tối thiểu là một mã băm trỏ đến tệp metainfo trên cổng tracker tập trung của bạn (còn gọi là your-tracker-portal.local).
- Tệp Metainfo: (còn được gọi là tệp .torrent) chứa tất cả các chi tiết về torrent của bạn, bao gồm địa chỉ của tracker (địa chỉ IP), độ dài của mỗi mảnh (piece length), và số lượng mảnh (piece-count).
- Các mảnh (Pieces): được chỉ định trong tệp Metainfo. Các mảnh là những phần có kích thước bằng nhau của bản tải về. Kích thước thông thường của mỗi mảnh là khoảng 512KB.
- Các tệp (Files): cũng được chỉ định trong tệp Metainfo. Có thể có nhiều tệp trong một torrent. Điều này có nghĩa là bạn có thể cần ánh xạ không gian địa chỉ của các mảnh vào không gian địa chỉ của tệp nếu có N mảnh và M tệp. Rất dễ mắc phải các lỗi tính toán đơn giản ở đây, nhưng bạn PHẢI thực hiện tất cả các bước ánh xạ địa chỉ bằng tay và tự mình định nghĩa cho các tham chiếu sau này. Để bắt đầu, bạn có thể triển khai một tệp mỗi torrent như là yêu cầu tối thiểu, nhưng nếu chỉ dừng ở bước này có thể sẽ dẫn đến điểm đánh giá thấp hơn.

3 Các chức năng chính của ứng dụng

3.1 Tracker Server

1. Chức năng quản lý metainfo file

Máy chủ hoạt động như một trung tâm điều phối trong hệ thống chia sẻ tệp, có nhiệm vụ giám sát các kết nối của client và các tệp mà các client đó sở hữu.

Máy chủ duy trì một Magnet text chứa các mã hash trỏ tới các metainfo file, cũng như tên và mô tả của các file này. Máy chủ ngoài chức năng tracker, còn đóng vai trò như một search engine dành cho các metainfo file, người dùng có thể thực hiện tìm kiếm và tải các metainfo file từ server.

2. Lưu trữ danh sách các client kết nối và các tệp của họ

Chức năng này giúp server quản lý các tệp trên máy chủ của mình. Khi một client kết nối, server sẽ lưu trữ thông tin về địa chỉ IP, cổng kết nối (Port), và trạng thái của client đó. Điều này giúp server có thể theo dõi và quản lý các client, đồng thời ngăn chặn các hành vi tấn công hoặc giả mạo địa chỉ IP.

Server cũng tạo một danh sách các tệp mà client đã công bố thông qua lệnh publish. Danh sách này chứa thông tin về các tệp, chẳng hạn như tên tệp, kích thước, thông tin các mảnh (pieces), mã hash của mỗi mảnh, tên máy (hostname) của client đang lưu trữ tệp, cùng với địa chỉ IP và cổng kết nối của client đó. Nhờ theo dõi các thông tin này, server có thể dễ dàng trả lời yêu cầu của các client khác về danh sách các peer lưu trữ tệp mà họ cần.

3. Trả lời yêu cầu của các client

Kiến trúc hệ thống hoạt động theo mô hình Client-Server. Cụ thể, các bước trong quy trình xử lý yêu cầu giữa client và server như sau:

- **Client gửi yêu cầu:** Client sẽ gửi một yêu cầu đến server, có thể là yêu cầu tải tệp (download) hoặc yêu cầu công bố tệp (publish).
- **Server nhận yêu cầu:** Server tiếp nhận yêu cầu từ client và xử lý theo loại yêu cầu, dữ liệu được gửi kèm và thông tin xác thực của client.
- **Server xử lý yêu cầu:** Server thực hiện các thao tác cần thiết dựa trên yêu cầu của client, có thể là gửi lại tệp, thông tin hoặc thực hiện các bước khác liên quan.
- **Server tạo và gửi phản hồi:** Sau khi xử lý xong yêu cầu, server sẽ tổng hợp các thông tin cần thiết vào một phản hồi, bao gồm metadata và dữ liệu nếu có. Server sau đó gửi phản hồi này lại cho client thông qua giao thức thích hợp.
- **Client nhận phản hồi:** Client nhận phản hồi từ server, thực hiện các thao tác như lưu trữ tệp đã tải xuống hoặc xử lý kết quả theo yêu cầu.

3.2 Client

1. Connect

- Chức năng: Kết nối với server để tham gia hệ thống chia sẻ tệp.
- Cách thức hoạt động:
 - Khởi tạo kết nối tới server qua địa chỉ IP và cổng.
 - Gửi yêu cầu kết nối với thông tin ID và địa chỉ của client, cũng như các metainfo file mà người dùng sở hữu và vẫn còn lưu trữ.
 - Nhận phản hồi từ server xác nhận trạng thái kết nối (thành công/thất bại).
 - Duy trì kết nối để thực hiện các thao tác như tải xuống hoặc công bố tệp.

2. Create

- Chức năng: Tạo một metainfo file cho một hoặc nhiều file, chứa thông tin như danh sách tệp, độ dài mảnh (piece length), và các thông tin bổ sung.
- Cách thức hoạt động:
 - Nhận các tham số như tên metainfo file, danh sách tệp tin của metainfo file, độ dài mảnh, và mô tả.
 - Kiểm tra kích thước từng tệp tin và tính toán số lượng mảnh.
 - Tạo thông tin về các tệp (đường dẫn, kích thước) và tính hash SHA-1 cho từng mảnh.
 - Tổng hợp thông tin vào metadata, bao gồm infohash (SHA-1 của thông tin tổng thể).

- Ghi metadata vào tệp `metafile_status.json`.

3. Publish

- Chức năng: Tải metainfo file lên tracker server.
- Cách thức hoạt động:
 - Kiểm tra xem client đã kết nối với tracker server chưa.
 - Đọc thông tin metainfo file từ `metafile_status.json`.
 - Tạo request chứa thông tin của metainfo file (tên, thông tin, mô tả, info của metafile, infohash, số byte đã tải xuống) để gửi đến tracker server.
 - Gửi yêu cầu POST đến tracker server để đăng tải tệp torrent.
 - Nếu phản hồi từ server là thành công, cập nhật thông tin của trường announce trong `metafile_status.json`.

4. Search

- Chức năng: Tìm kiếm metainfo file trên tracker server dựa trên từ khóa.
- Cách thức hoạt động:
 - Tạo một yêu cầu chứa từ khóa tìm kiếm (**keyword**) gửi đến tracker server.
 - Gửi yêu cầu thông qua giao thức HTTP.
 - Nhận phản hồi từ server, bao gồm danh sách các tệp torrent phù hợp với từ khóa.
 - Trả về danh sách kết quả.

5. Get_torrent

- Chức năng: Nhận thông tin về một tệp torrent từ tracker server.
- Cách thức hoạt động:
 - Gửi yêu cầu GET đến tracker server với **infohash** của metainfo file cần lấy.
 - Nhận phản hồi từ server chứa thông tin của metainfo file.
 - Lưu thông tin nhận được vào `metafile_status.json`.

6. My_torrent

- Chức năng: Lấy danh sách các tệp torrent mà client đang sở hữu.
- Cách thức hoạt động:
 - Đọc tệp `metafile_status.json` để lấy thông tin về tất cả các tệp torrent mà client có.
 - Lọc các thông tin về tệp torrent, bao gồm **infohash**, mô tả, và số byte đã tải xuống.
 - Trả về danh sách các tệp torrent đang có.

7. Download

- Chức năng: Tải xuống các file thuộc metainfo file từ các peers.
- Cách thức hoạt động:

- Đọc thông tin tệp torrent từ `metafile_status.json`.
- Tạo yêu cầu gửi đến tracker server để bắt đầu quá trình tải.
- Lấy danh sách các peer từ tracker server.
- Kết nối với các peer, thực hiện handshake và tải xuống các piece của metainfo file theo chiến lược rarest-first. Ghi lại lịch sử download của từng piece ứng với từng peer tại các thời điểm khác nhau.
- Sau khi tải xong, cập nhật trạng thái đã tải xong và gửi yêu cầu hoàn tất tải xuống đến tracker server.

8. Upload

- Chức năng: Tải lên các piece thuộc metainfo file từ các peers.
- Cách thức hoạt động:
 - Nhận yêu cầu thiết lập kết nối trên giao thức bittorrent từ người dùng khác.
 - Cung cấp piece theo yêu cầu của các peer.
 - Ghi lại lịch sử upload từng piece ứng với từng peer tại các thời điểm.

3.3 Giao thức giao tiếp của ứng dụng

3.3.1 Server và client

Sử dụng giao thức HTTP để phục vụ giao tiếp giữa Server và client. Phản hồi các yêu cầu HTTP GET. Các yêu cầu bao gồm số liệu từ máy khách giúp trình theo dõi lưu giữ số liệu thống kê chung về torrent. Phản hồi bao gồm danh sách ngang hàng giúp máy khách tham gia vào torrent.

Các tham số của client gửi tới server:

- `info_hash`: mã hash SHA1 20 được mã hóa url của trường `info` từ tệp Metainfo, cần thiết khi lấy danh sách peer hoặc cập nhật thông tin của client.
- `id`: ID của client.
- `port`: Cổng nhận yêu cầu từ các peer khác của client, cung cấp khi thiết lập kết nối với tracker server.
- `downloaded`: Số byte đã tải ứng với metainfo file gửi tới server, cung cấp khi bắt đầu quá trình tải metainfo file từ các peer.
- `event`: `started` - cung cấp với request đầu tiên hoặc khi bắt đầu tải file từ peer. `Completed` - khi hoàn tất tải từ các peer đang hoạt động. `Stopped` - khi thoát ứng dụng.
- `ip`: Địa chỉ ip của client.

Tracker server phản hồi bằng "text/plain" document bao gồm một dictionary có các khóa sau:

- `Failure reason`: Nếu có, thì không cần thiết phải có các khóa khác. Giá trị là thông báo lỗi về lý do xử lý request không thành công.
- `Tracker id`: Chuỗi mà máy khách phải gửi lại trong thông báo tiếp theo. Nếu không có và thông báo trước đó đã gửi Tracker ID, không loại bỏ giá trị cũ; hãy tiếp tục sử dụng.

- Peers: Danh sách dictionary của các peer với các khóa: peer id, ip và port.

Ứng với mỗi chức năng, các tham số được gửi có thể khác nhau, nhưng chủ yếu dựa vào các tham số chính phía trên. Tương tự với phản hồi từ server.

3.3.2 Client và client

Sử dụng giao thức Bittorrent dựa trên TCP/IP cho tầng vận chuyển.

- Handshake: Handshake là một thông điệp bắt buộc và phải là thông điệp đầu tiên được truyền đi bởi máy khách. Thông điệp này dài $(49 + \text{len}(\text{pstr}))$ byte. Cấu trúc của thông điệp: $\langle \text{pstrlen} \rangle \langle \text{pstr} \rangle \langle \text{reserved} \rangle \langle \text{info_hash} \rangle \langle \text{peer_id} \rangle$.
 - pstrlen: Chiếm 1 byte, thể hiện độ dài của $\langle \text{pstr} \rangle$
 - pstr: chuỗi định danh của giao thức.
 - reserved: 8 byte, thường có giá trị 0 với hầu hết các ứng dụng bittorrent hiện nay.
 - info_hash: 20-byte SHA1 hash của khóa info thuộc metafile.
 - peer_id: Id của client, thường là 20 bytes.

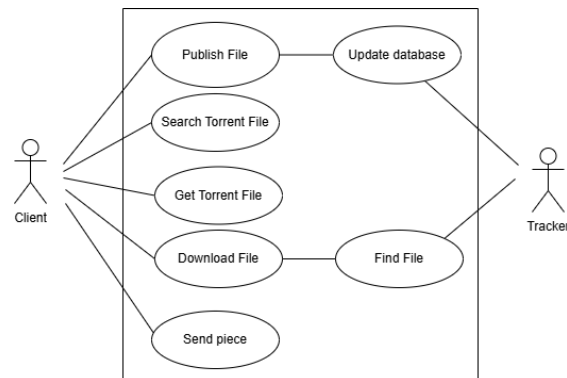
Các thông điệp (message) giao tiếp giữa client và client. Các thông điệp này đều có cấu trúc $\langle \text{length prefix} \rangle \langle \text{message ID} \rangle \langle \text{payload} \rangle$. Trong đó length chiếm 4 byte, id chiếm 1 byte, và payload sẽ phụ thuộc vào kiểu thông điệp.

- have: $\langle \text{len}=0005 \rangle \langle \text{id}=4 \rangle \langle \text{piece index} \rangle$. Thông báo cho peer rằng người gửi đã tải và xác nhận thành công piece có index được gửi.
- bitfield: $\langle \text{len}=0001+X \rangle \langle \text{id}=5 \rangle \langle \text{bitfield} \rangle$. Được gửi ngay sau khi thực hiện handshake, thông báo cho người tải biết peer hiện đang sở hữu các piece nào ứng với metainfo file người tải yêu cầu.
- request: $\langle \text{len}=0013 \rangle \langle \text{id}=6 \rangle \langle \text{index} \rangle \langle \text{begin} \rangle \langle \text{length} \rangle$. Yêu cầu các block bắt đầu từ begin với độ dài length của piece nào đó từ peer khác. Để đơn giản, ứng dụng của nhóm sẽ yêu cầu và trả tất cả byte của từng piece chứ không chia nhỏ piece thành nhiều block.
- piece: $\langle \text{len}=0009+X \rangle \langle \text{id}=7 \rangle \langle \text{index} \rangle \langle \text{begin} \rangle \langle \text{block} \rangle$. Trả về một block data ứng với yêu cầu bên trên.

4 Hiện thực ứng dụng

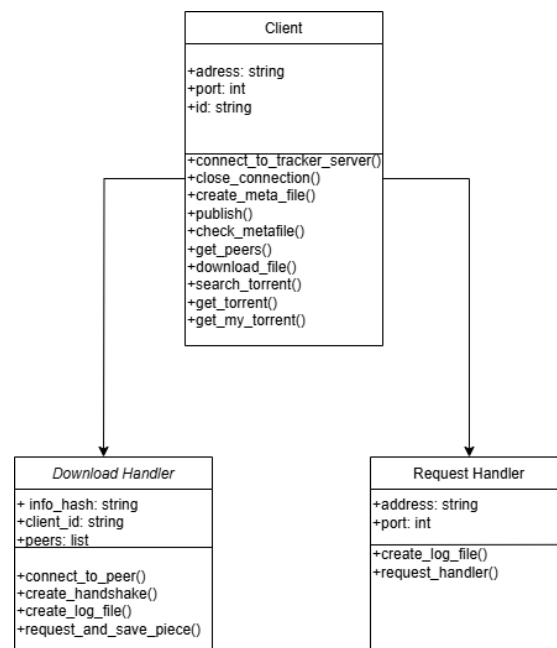
4.1 Diagram

4.1.1 Use-case diagram



Hình 1: Lược đồ use-case

4.1.2 Class diagram

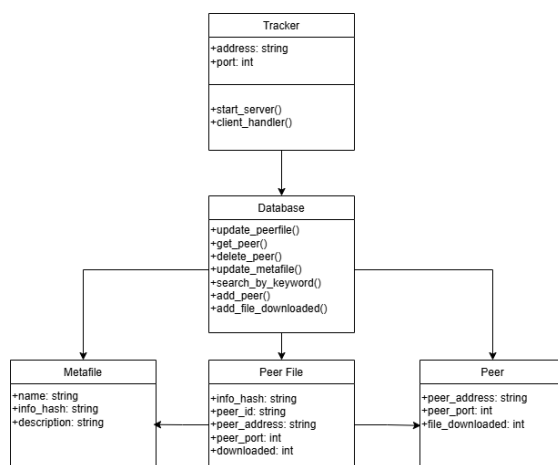


Hình 2: Lược đồ class ở phía Client

Ở phía client, sẽ có các hàm phục vụ cho các chức năng chính của ứng dụng.

- `connect_to_tracker_server`: Thực hiện kết nối tới tracker server thông qua địa chỉ được cung cấp, sau khi kết nối, gửi yêu cầu kèm theo magnet text của các metainfo file đang sở hữu tới server để server cập nhật và thêm peer vào mạng.
- `close_connection`: Gửi yêu cầu với event stopped tới server. Kết thúc ứng dụng.
- `create_meta_file`: Nhận vào tên, danh sách file, mô tả, piece length, tạo và lưu file vào thư mục hoạt động.
- `publish`: Gửi yêu cầu publish file tới server.
- `check_metafile`: Kiểm tra tình trạng metainfo file.
- `get_peers`: Gửi yêu cầu với `info_hash` tới server, nhận và trả về danh sách các peer.
- `download_file`: Sử dụng lớp download handler, thực hiện gửi cập nhật tới server và tải file từ các peer.
- `search_torrent`: Gửi request tìm kiếm torrent theo keyword tới server và trả về danh sách torrent phù hợp.
- `get_torrent`: Gửi yêu cầu tới server để nhận metainfo của info hash gửi đi. Lưu lại metainfo file.
- `get_my_torrent`: Trả về các torrent đang sở hữu.

Các lớp download và request handler thực hiện các công việc download và uploading.



Hình 3: Lược đồ class ở phía Tracker

Server nhận và xử lý các yêu cầu của người dùng. Sử dụng lớp database để xử lý các tác vụ liên quan đến lưu trữ. Có các bảng chính là:

- **Metafile**: Đóng vai trò như magnet text, chứa các max info_hash trỏ tới các metainfo file lưu trên máy chủ. Ngoài ra còn lưu tên và mô tả phục vụ cho mục đích tìm kiếm.
- **PeerFile**: Lưu trữ peer nào đang sở hữu file nào, số byte sở hữu, địa chỉ và cổng nhận yêu cầu của peer, phục vụ cho các yêu cầu get peer.
- **Peer**: Lưu thông tin thống kê số file tải của các peer.

4.2 Cấu trúc metainfo file

Một metainfo file có cấu trúc như sau:

```
1 {  
2   'announce': '192.168.59.133',  
3   'info':  
4     {  
5       'files':  
6         [  
7           {'length': 120, 'path': ['text1.txt']},  
8           {'length': 240, 'path': ['cat.png']}  
9         ],  
10      'name': 'metafile',  
11      'piece length': 4096,  
12      'pieces': <SHA1 hashes hex string>  
13    }  
14 }
```

Trong đó:

- announce: Địa chỉ của tracker sở hữu file.
- info: Trường chứa thông tin chính của metainfo file. Gồm:
 - files: Danh sách các file mà metainfo file bao gồm. Danh sách gồm các dictionary chứa khóa về độ dài và đường dẫn của các file.
 - name: tên của metafile.
 - piece length: Độ dài theo byte của mỗi piece.
 - pieces: Một chuỗi được nối lại từ các mã hash của các piece theo thứ tự từ nhỏ đến lớn. Được sử dụng để kiểm tra tính đúng đắn của các piece nhận từ các peer bên ngoài.

Mapping giữa piece và file: Để biết piece nào thuộc file nào, ta có thể thực hiện một phép chiếu đơn giản: các piece từ 1 đến $\lceil \frac{file1_length}{piece_length} \rceil$ thuộc file1, các piece từ $\lceil \frac{file1_length}{piece_length} \rceil + 1$ đến $\lceil \frac{file1_length}{piece_length} \rceil + \lceil \frac{file2_length}{piece_length} \rceil$ thuộc file2, và tiếp tục như thế.

Quản lý các metainfo file ở phía client: Đối với client, ta sẽ có một magnet text là metafile_status.json quản lý các metainfo file thông qua infohash (mã hash của khóa info), có các khóa như mô tả, các piece sở hữu, số byte đã download ứng với mỗi metainfo file. Còn server sẽ chỉ lưu trữ infohash cũng như info của các metafile.

4.3 Hiện thực downloading

Trước khi tải, ta gửi và nhận danh sách các peer đang sở hữu file từ server. Sau đó gửi một yêu cầu download tới server để thông báo mình bắt đầu thực hiện quá trình tải file, và đồng thời trở thành một peer sẵn sàng upload file này.

Quá trình tải file gồm các bước chính là, thiết lập kết nối và nhận thông tin từ các peer, lên chiến lược chọn peer để yêu cầu ứng với từng piece. Tải và lưu lại các piece vào file vật lý trên máy.

4.3.1 Thiết lập kết nối

Ở bước thiết lập, sử dụng đa luồng để thiết lập các kết nối một cách đồng thời tới các peer, xác định giao thức và cập nhật bitfield của các peer. Qua đó xây dựng được danh sách các peer ứng với từng piece.

Ta cũng ghi lại thời điểm kết nối với từng peer vào file log phục vụ cho mục đích thống kê.

278 10:00:50.383912 192.168.54.31	192.168.54.250	BitTor...	132 Handshake	Continuation data
> Frame 278: 132 bytes on wire (1056 bits), 132 bytes captured (1056 bits) on interface \Device\NPF...				
> Ethernet II, Src: Intel_fe:85:e4 (48:f1:7f:fe:85:e4), Dst: AzureWaveTec_67:e3:d3 (b4:8c:9d:67:e3:d3)				
> Internet Protocol Version 4, Src: 192.168.54.31, Dst: 192.168.54.250				
> Transmission Control Protocol, Src Port: 52853, Dst Port: 3000, Seq: 1, Ack: 1, Len: 78				
> BitTorrent				
> BitTorrent				

0000	b4 8c 9d 67 e3 d3 48 f1 7f fa 85 e4 08 00 45 00	g H
0010	00 76 1b ad 40 00 00 06 f0 6a c0 a8 36 1f c0 a8	v @ ... j 6 ...
0020	36 fa ce 75 0b b8 1a e6 fa 66 54 14 1e 64 50 18	6 u ... FT dP...
0030	00 ff ec 70 00 00 13 42 69 74 54 6f 72 72 65 6e	... B itTorren
0040	74 20 70 72 6f 74 6f 63 6f 6c 00 00 00 00 00	t protoc ol.....
0050	00 00 32 61 34 30 35 64 36 66 34 61 61 36 34 32	--2a405d 6f4aa642
0060	31 38 38 39 66 35 64 33 30 62 38 39 64 30 33 34	1889f5d3 0089d034
0070	62 31 62 65 31 30 61 31 32 65 59 4c 36 41 32 38	b1bel0a1 2eVL6A28
0080	41 59 39 57	AY9W

Hình 4: Ví dụ về handshake được gửi tới peer.

4.3.2 Chiến lược tải file Rarest-First

Ở đây, nhóm sử dụng chiến lược tải Rarest-First. Cụ thể như sau:

- Sắp xếp lại thứ tự các piece theo số lượng peer sở hữu piece đó, số lượng này được gọi là rarity của piece. Rarity càng thấp chứng tỏ piece này càng hiếm. Ta sẽ ưu tiên những piece hiếm này trước. Mục đích của việc này là khiến những piece hiếm này sẽ dần trở nên phổ biến, giúp các client khác tải file cùng thời điểm có thể tải từ nhiều peer hơn. Đồng thời cũng giúp quá trình tải file đảm bảo hơn, tránh trường hợp các peer sở hữu những piece hiếm ngừng quá trình upload.
- Đối với các piece có cùng độ hiếm, ta xáo trộn ngẫu nhiên thứ tự các piece này với nhau, vì nếu để nguyên thứ tự của thuật toán sắp xếp, tất cả client đang tải file đều sẽ đồng loạt gửi request đến những peer sở hữu piece hiếm nhất, điều này khiến cho peer đó bị quá tải dẫn đến hiệu suất tổng thể giảm. Việc thêm yếu tố ngẫu nhiên vào thứ tự tải giúp chia đều công việc cho các peer sở hữu piece cùng độ hiếm.
- Cuối cùng, lần lượt gửi yêu cầu theo thứ tự các piece đã sắp xếp, ta sẽ sử dụng đa luồng với số luồng nhất định để xử lý các phản hồi một cách song song. Khi nhận piece, ta dùng SHA1 để hash nội dung piece đó, kiểm tra với mã hash đã được lưu trong metainfo file, nếu hợp lệ thì lưu lại. Cuối cùng là gửi thông điệp have cho peer để cập nhật rằng ta đã sở hữu piece.

Trong quá trình tải, sau mỗi 5 phút, client sẽ gửi một request tới server để cập nhật số byte đã tải và để nhắc server rằng bản thân client vẫn là một peer đang hoạt động và upload file này.

Sau khi tải tất cả piece, các file được lưu vào một thư mục cố định. Người dùng có thể mở các file được tải từ metainfo file. Cuối cùng, gửi yêu cầu hoàn tất tải file tới server.

Quá trình tải file cũng được ghi lại vào file log.

4.3.3 Tải đồng thời nhiều file

Với mỗi yêu cầu tải file từ người dùng, ứng dụng sẽ tạo một luồng thực thi mới và thực hiện tải. Sau khi tải hoàn tất, ứng dụng thông báo cho người dùng và ghi lại lịch sử vào file log. Điều này cho phép người dùng thực hiện các chức năng khác trong quá trình tải, hoặc tải nhiều file cùng lúc.

```
>> download 4459aff372d3deda6aa87035b993dca173277e5a
Downloading from [{"peerid": "0987UNZW09", "addr": "127.0.0.1", "port": 3005}] ... We'll notice you when the download is complete!
>> download a1dd25ad8bc02221b5b7fbaa8d404a39c4cb318
Downloading from [{"peerid": "0C8A5BX5EN", "addr": "127.0.0.1", "port": 3000}, {"peerid": "0987UNZW09", "addr": "127.0.0.1", "port": 3005}] ... We'll notice you when t
he download is complete!
>>
Finish download: a1dd25ad8bc02221b5b7fbaa8d404a39c4cb318, you can see log file of download process!
>> my torrent
1. a1dd25ad8bc02221b5b7fbaa8d404a39c4cb318, Downloaded: 196688, Description: test
2. 4459aff372d3deda6aa87035b993dca173277e5a, Downloaded: 0, Description: file from Huy
>>
Finish download: 4459aff372d3deda6aa87035b993dca173277e5a, you can see log file of download process!
>> |
```

Hình 5: Thực hiện nhiều lệnh download đồng thời.

4.4 Hiện thực uploading

Đối với việc tải file, nhóm sử dụng đa luồng để tạo nhiều luồng xử lý yêu cầu từ các peer khác một cách đồng thời. Qua đó xử lý đồng thời yêu cầu từ nhiều peer khác nhau.

Khi một yêu cầu được gửi tới, một luồng thực thi được tạo ra để xử lý yêu cầu. Sau khi kết thúc quá trình, luồng thực thi kết thúc.

Cụ thể, ứng dụng sẽ kiểm tra giao thức giao tiếp bittorrent từ các peer khác. Sau khi thiết lập kết nối, ứng dụng sẽ gửi bitfield tới peer gửi yêu cầu. Đối với thông điệp yêu cầu piece, ứng dụng truy cập vào thư mục chứa file để lấy piece tương ứng và gửi trả cho peer yêu cầu. Sau khi nhận được thông điệp have, ứng dụng kết thúc kết nối.

Quá trình upload từng piece cho peer cụ thể được lưu lại trong suốt phiên hoạt động của ứng dụng.

4.5 Hiện thực các chức năng khác

Đối với chức năng create. Người dùng chỉ cần nhập vào tên, các file, mô tả và piecelenght, sau đó ứng dụng sẽ xử lý lần lượt các file trong danh sách, kiểm tra kích thước và tính số lượng piece ứng với từng file. Lưu tại vào metainfo_status.

Đối với publish, người dùng nhập vào info_hash, ứng dụng truy cập và lấy thông tin của metainfo file ứng với mã hash đó, gửi cho server để cập nhật, và người dùng sẽ trở thành seeder đầu tiên nếu trước đó không tồn tại metainfo file tương tự.

Đối với các chức năng search và get torrent, người dùng gửi yêu cầu tới server, server sẽ trả lại metainfo file tương ứng và ứng dụng sẽ lưu lại metainfo file đó.

5 Kết quả hiện thực

Ta kiểm thử các chức năng của hệ thống trên một mạng P2P nhỏ gồm 3 máy khách, 1 máy chủ đóng vai trò tracker. Các máy đều được kết nối cùng một mạng cục bộ và sử dụng địa chỉ IPv4 để kết nối với nhau.

Server sẽ chạy ở địa chỉ 192.168.54.250, cổng 6888.

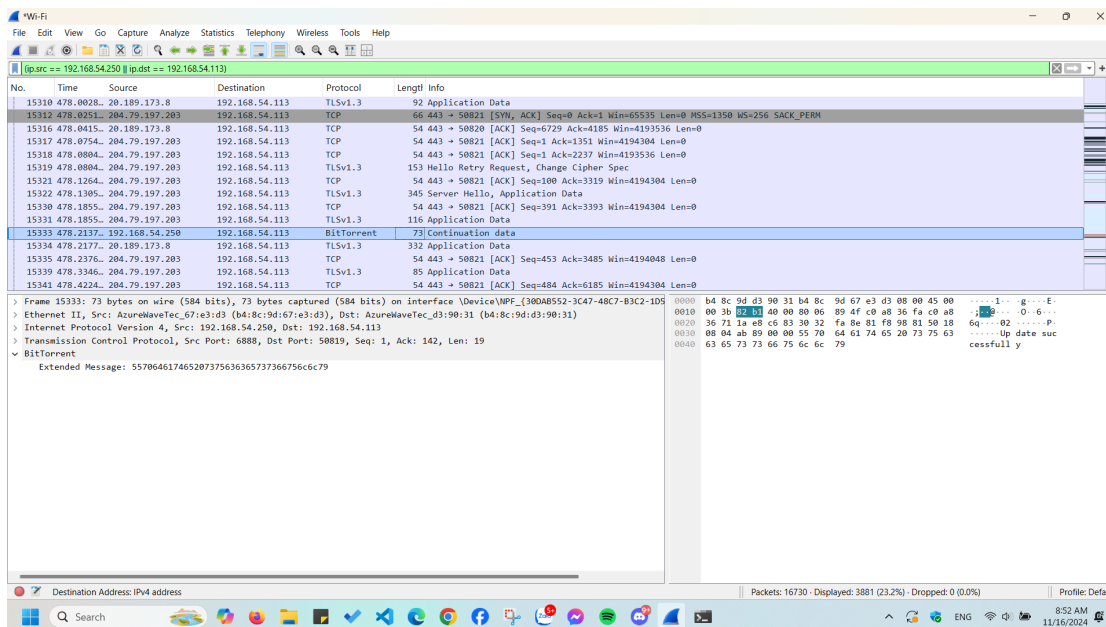
Máy khách sẽ gồm 3 địa chỉ: 192.168.54.250, 192.168.54.113 và 192.168.54.31.

5.1 Chức năng Connect

Chúng ta sẽ khởi tạo kết nối tới server qua địa chỉ IP 192.168.54.250 và cổng 6888 từ máy khách có địa chỉ 192.168.54.113. Hình ảnh dưới đây thể hiện được trạng thái kết nối thành công và được ghi nhận trong phần mềm Wireshark sau khi ta đã gửi magnet text tới server.

```
Type 'connect [hostname] [port]' to connect to other tracker server!  
>> connect 192.168.54.250 6888  
Connect successfully!
```

Hình 6: Khởi tạo kết nối



Hình 7: Kết nối được ghi nhận trong Wireshark, server thực hiện cập nhật thông tin của peer thành công

5.2 Chức năng Create

Chúng ta tiến hành tạo metainfo file dựa trên các file cho trước, sau đó kiểm tra thông tin của metadata trong file `metainfo_status.json`.

```
>> create  
>>> Name: file  
>>> Files: img1.png img2.png  
>>> Description: Test create function  
>>> Piece length: 4096  
Processing file: file\img1.png  
Processing file: file\img2.png  
Metainfo created successfully!  
InfoHash: d9e32a4b7040ceb4acef580510d41d23314a42de  
>> |
```

Hình 8: Tạo metainfo file dựa trên file img1.png

[illegible]

Hình 9: Kiểm tra thông tin metadata trong file metafile_status.json

```
"info": {
  "files": [
    {
      "length": 96318,
      "path": "img1.png"
    },
    {
      "length": 96318,
      "path": "img2.png"
    }
  ],
  "name": "file",
  "piece_length": 4096,
  "pieces":
    "7c2547c64d3639e604931b41c48e976108ddc3e43fb7d08ec11925286e78e8dc30a382d35c62104a0c7"
```

Hình 10: Khóa info của metainfo file.

5.3 Chức năng Publish

Chức năng này cho phép ta publish các metainfo file đã tạo trên máy bằng cách truyền vào info hash. Nếu không truyền vào tham số, ta sẽ được yêu cầu tạo lại metainfo file từ đầu để thực hiện publish.

Chúng ta sẽ tiến hành publish một metainfo file gồm 2 file là `cat1.png` và `cat2.png`. Sau đó chúng ta sẽ xác nhận việc gửi thành công thông qua hệ quản trị cơ sở dữ liệu PostgreSQL của server.

```
>> publish
>>> Name: Cat image
>>> Files: cat1.png cat2.jpg
>>> Description: cute cat pic
>>> Piece length: 16384
Processing file: file\cat1.png
Processing file: file\cat2.jpg
Publish file successfully
>>
```

Hình 11: Các dòng lệnh để tạo và publish metainfo file gồm file cat1.png và cat2.png

☐	name	infohash	description	peerfiles
☐	Cat_image	2a405d6f4aa6421889f5d3...	Random image from Long	peerfiles
☐	Cat image	9f4e8ef4d4398dba7d2b83...	cute cat pic	peerfiles
☐	Image file	a1dd25ad0bc022221b5b7f...	test	peerfiles

Hình 12: Thông tin các file được gửi ở PostgreSQL DBMS

Database ở phía server đã lưu lại info hash cũng như tên và mô tả của metainfo file phía trên ta mới tạo. Ở bảng dưới, người tải lên là (192.168.54.250:3000) được ghi nhận là đang seeding cho file này ở bảng peerfile thể hiện quan hệ peer và file.

id	infohash	peerid	peeraddr	peerport	downloaded	metafile
245	2a405d6f4aa6421889f5d3...	PH0IPRG76T	192.168.54.113	3000	196608	metafile
249	9f4e8ef4d4398dba7d2b83...	1050WJ15H8	192.168.54.250	3000	278528	metafile

name	infohash	description
Cat image	9f4e8ef4d439...	cute cat pic

Hình 13: Thông tin các file được gửi ở PostgreSQL DBMS

5.4 Chức năng Search

Tiếp theo, chúng ta sẽ tìm kiếm tệp torrent **image** trên tracker server dựa trên từ khóa.

```
>> search image
1. 2a405d6f4aa6421889f5d30b89d034b1be10a12e. Description: Random image from Long
2. b94756e07fbdbb1e2deac348aec993cce4d8b56d. Description: image about cat
>> 
```

Hình 14: Tìm kiếm tệp torrent **image**

5.5 Chức năng Get_torrent

Chúng ta sẽ tiến hành nhận thông tin về tệp torrent **image** từ tracker server dựa trên info hash, sau đó kiểm tra trong `metafile_status.json`.

```
>> get_torrent 2a405d6f4aa6421889f5d30b89d034b1be10a12e
Success!
```

Hình 15: Dòng lệnh lấy tệp torrent dựa trên hash keyword


```
"description": "Random image from Long",  
"piece_have": "111111111111111111111111",  
"downloaded": 196608,  
"infohash": "2a405d6f4aa6421889f5d30b89d034b1be10a12e"
```

Hình 16: Thông tin nhận được trong metafile_status.json

```
"info": {  
  "files": [  
    {  
      "length": 96318,  
      "path": "img1.png"  
    },  
    {  
      "length": 96318,  
      "path": "img2.png"  
    }  
  ],  
  "name": "Cat_image",  
  "piece_length": 8192,  
  "pieces": "2dfe4d4a3f69a38b780073a256f6773a17"
```

Hình 17: Info của metainfo file nhận được

5.6 Chức năng My_torrent

Chúng ta tiến hành gọi hàm để trả về danh sách các tệp torrent mà client đang sở hữu.

```
>> my_torrent  
1. a1dd25ad0bc022221b5b7fbba8d404a39c4cb318. Downloaded: 0. Description: test  
2. 4459aff372d3deda6aa87035b993dca173277e5a. Downloaded: 171704320. Description: file from Huy  
3. 12abe4a9b956fc3acd9e3161383774c16aa42e6. Downloaded: 97280. Description: Test create function  
4. d9e32a4b7040ceb4acef580510d41d23314a42de. Downloaded: 196608. Description: Test create function  
5. 2a405d6f4aa6421889f5d30b89d034b1be10a12e. Downloaded: 0. Description: Random image from Long  
>>
```

Hình 18: Danh sách các tệp torrent mà client đang sở hữu

5.7 Chức năng Download

Dựa trên info hash trên, chúng ta tiến hành tải các file, sau đó kiểm tra trạng thái của quá trình tải trên Wireshark, file log và mở file đã tải để xác nhận.

Ta sẽ thực hiện tải một torrent gồm 2 file ảnh, thực hiện tải xuống máy có địa chỉ 192.168.54.31. Peer đang sở hữu file có địa chỉ 192.168.54.250 (cùng địa chỉ với server do chạy trên cùng một máy, khác listen port).



```
>> download 2a405d6f4aa6421889f5d30b89d034b1be10a12e
Downloading from [{"peerid": "QTBUGIZKH", "addr": "192.168.54.250", "port": 3000}] ... We'll notice you when the download is complete!
>>
>>
Finish download: 2a405d6f4aa6421889f5d30b89d034b1be10a12e, you can see log file of download process!
>> |
```

Hình 19: Tải xuống các tệp dựa trên info hash

262 10:00:47.531723 192.168.54.31	192.168.54.250	BitTor...	196 Continuation data
263 10:00:47.717804 192.168.54.250	192.168.54.31	TCP	54 6888 → 52016 [ACK] Seq=1395 Ack=835 Win=2053 Len=0
> Frame 262: 196 bytes on wire (1568 bits), 196 bytes captured (1568 bits) on interface \Device\NPF... 192.168.54.31			
> Ethernet II, Src: Intel_fe:85:e4 (48:f1:7f:fe:85:e4), Dst: AzureWaveTec_67:e3:d3 (b4:8c:9d:67:e3:d3)			
> Internet Protocol Version 4, Src: 192.168.54.31, Dst: 192.168.54.250			
> Transmission Control Protocol, Src Port: 52016, Dst Port: 6888, Seq: 693, Ack: 1395, Len: 142			
> BitTorrent			
	0000	b4 8c 9d 67 e3 d3 48 f1 7f fe 85 e4 08 00 45 00	...g...H...E...
	0010	00 b6 1b a8 40 00 80 06 f0 2f c0 a8 36 1f c0 a8	...@.../...6...
	0020	36 fa cb 30 1a e8 af 4a 18 67 7c a6 84 47 50 18	6...0...J...g...GP...
	0030	00 ff 6a 0d 00 00 47 45 54 20 2f 20 48 54 50 50	...f...GE T / HTTP
	0040	2f 31 2e 31 0d 0a 48 6f 73 74 3a 31 39 32 2e 31	/1.1...Ho st:192.1
	0050	56 30 2e 35 34 2e 33 31 2f 70 65 65 72 73 0d 0a	68.54.31 /peers...
	0060	0d 0a 70 22 61 63 74 69 6f 6e 22 3a 20 22 67 65	...{"acti on": "ge
	0070	74 2d 70 65 65 72 22 2c 20 22 69 64 22 3a 20 22	t-peer", "id": "
	0080	59 4c 36 41 32 38 41 59 39 57 22 2c 20 22 69 6e	YL6A28AY 9w", "in
	0090	66 6f 68 61 73 68 22 3a 20 22 32 61 34 30 35 64	fohash": "2a405d
	00a0	36 66 34 61 61 36 34 32 31 38 38 39 66 35 64 33	6f4aa642 1889f5d3
	00b0	30 62 30 39 64 30 33 34 62 31 62 65 31 30 61 31	0b89d034 b1be10a1
	00c0	72 65 22 7c	2e"]

Hình 20: Yêu cầu GET peer gửi tới server được ghi nhận trong Wireshark

> Frame 264: 184 bytes on wire (1472 bits), 184 bytes captured (1472 bits) on interface \Device\NPF... 192.168.54.31	0000	48 f1 7f fe 85 e4 b4 8c 9d 67 e3 d3 08 00 45 00	H...:... g...E...
> Ethernet II, Src: AzureWaveTec_67:e3:d3 (b4:8c:9d:67:e3:d3), Dst: Intel_fe:85:e4 (48:f1:7f:fe:85:e4)	0010	00 aa fd ee 40 00 80 06 0d f5 c0 a8 36 fa c0 a8	...@.../...6...
> Internet Protocol Version 4, Src: 192.168.54.250, Dst: 192.168.54.31	0020	36 1f 1a e8 cb 30 7c a6 84 47 af 4a 18 f5 50 18	6...0...J...G...P...
> Transmission Control Protocol, Src Port: 6888, Dst Port: 52016, Seq: 1395, Ack: 835, Len: 130	0030	08 05 ae 41 00 00 7b 22 73 74 61 74 75 73 22 3a	...A...[" status":
> BitTorrent	0040	20 22 73 75 63 63 65 73 73 22 2c 20 22 74 72 61	...succes s", "tra
	0050	63 6b 65 72 5f 69 64 22 3a 20 22 31 39 32 2e 31	cker_id": "192.1
	0060	36 30 2e 35 34 2e 32 35 30 22 2c 20 22 70 65 69	68.54.25 0", "pee
	0070	72 73 22 3a 20 5b 70 22 70 65 65 72 69 64 22 3a	rs": [{"peerid":
	0080	20 22 37 52 4c 4e 59 4e 48 59 37 53 22 2c 20 22	"7RLUVN HY75", "
	0090	61 64 64 72 22 3a 20 22 31 39 32 2e 31 36 38 2e	addr": " 192.168.
	00a0	35 34 2e 32 35 30 22 2c 20 22 70 6f 72 74 22 3a	54.250", "port":
	00b0	20 33 30 30 30 7d 5d 7d	3000]]]

Hình 21: Danh sách peer nhận được từ server

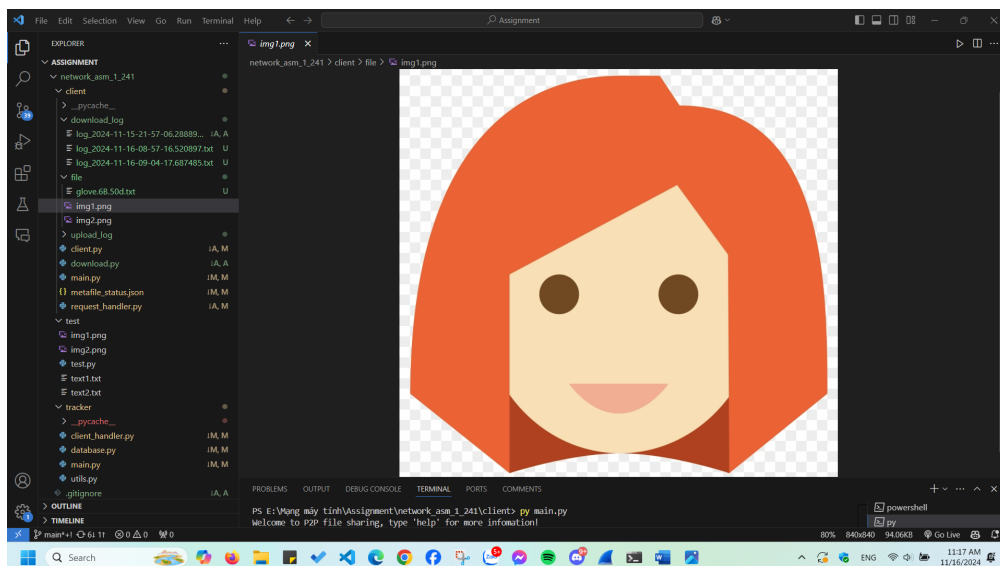
278 10:00:50.383912 192.168.54.31	192.168.54.250	BitTor...	132 Handshake Continuation data
> Frame 278: 132 bytes on wire (1056 bits), 132 bytes captured (1056 bits) on interface \Device\NPF... 192.168.54.31			
> Ethernet II, Src: Intel_fe:85:e4 (48:f1:7f:fe:85:e4), Dst: AzureWaveTec_67:e3:d3 (b4:8c:9d:67:e3:d3)			
> Internet Protocol Version 4, Src: 192.168.54.31, Dst: 192.168.54.250			
> Transmission Control Protocol, Src Port: 52853, Dst Port: 3000, Seq: 1, Ack: 1, Len: 78			
> BitTorrent			
> BitTorrent			
	0000	b4 8c 0d 67 e3 d3 48 f1 7f fe 85 e4 08 00 45 00	...g...H...E...
	0010	00 76 1b ad 40 00 80 06 f0 6a c0 a8 36 1f c0 a8	...v...@...j...6...
	0020	36 fa ec 75 0b 8b 1a e6 fa 66 54 14 1e 64 50 18	6...u...-FT...dP...
	0030	00 ff ec 70 00 00 13 42 69 74 54 6f 72 72 65 6e	...p...B itTorren
	0040	74 20 70 72 6f 74 6f 63 6f 6c 00 00 00 00 00 00	t protac ol:-----
	0050	00 00 32 61 34 30 35 64 36 66 34 61 61 36 34 32	...2a405d 6f4aa642
	0060	31 38 38 39 66 35 64 33 30 62 38 39 64 30 33 34	1889f5d3 0b89d034
	0070	62 31 62 65 31 30 61 31 32 65 59 4c 36 41 32 38	b1be10a1 2eYL6A28
	0080	41 59 39 57	AY9W

Hình 22: Quá trình handshake giữa các peer

Theo sau đó là quá trình tải và nhận piece được ghi lại trong file log sau.

```
1 2024-11-16 09:04:17.687485: Start downloading, meatafile: 2a405d6f4aa6421889f5d30b89d034b1be10a12e
2 2024-11-16 09:04:17.896378: Establish connection to peer I341JF1JHT-192.168.54.250:3000
3 2024-11-16 09:04:17.896378: Establish connection to peer QTBUGIZKHW-192.168.54.250:3000
4 2024-11-16 09:04:17.931051: Successfully download piece 1 belong to img1.png from 192.168.54.250:3000
5 2024-11-16 09:04:17.931051: Successfully download piece 2 belong to img1.png from 192.168.54.250:3000
6 2024-11-16 09:04:17.931051: Successfully download piece 0 belong to img1.png from 192.168.54.250:3000
7 2024-11-16 09:04:17.942148: Successfully download piece 5 belong to img1.png from 192.168.54.250:3000
8 2024-11-16 09:04:17.949160: Successfully download piece 3 belong to img1.png from 192.168.54.250:3000
9 2024-11-16 09:04:18.012526: Successfully download piece 4 belong to img1.png from 192.168.54.250:3000
10 2024-11-16 09:04:18.018028: Successfully download piece 9 belong to img1.png from 192.168.54.250:3000
11 2024-11-16 09:04:18.021320: Successfully download piece 8 belong to img1.png from 192.168.54.250:3000
12 2024-11-16 09:04:18.021320: Successfully download piece 7 belong to img1.png from 192.168.54.250:3000
13 2024-11-16 09:04:18.021320: Successfully download piece 6 belong to img1.png from 192.168.54.250:3000
14 2024-11-16 09:04:18.034865: Successfully download piece 10 belong to img1.png from 192.168.54.250:3000
15 2024-11-16 09:04:18.050608: Successfully download piece 11 belong to img1.png from 192.168.54.250:3000
16 2024-11-16 09:04:18.051608: Successfully download piece 12 belong to img1.png from 192.168.54.250:3000
17 2024-11-16 09:04:18.051608: Successfully download piece 13 belong to img2.png from 192.168.54.250:3000
18 2024-11-16 09:04:18.056587: Successfully download piece 14 belong to img2.png from 192.168.54.250:3000
19 2024-11-16 09:04:18.063687: Successfully download piece 17 belong to img2.png from 192.168.54.250:3000
20 2024-11-16 09:04:18.124074: Successfully download piece 15 belong to img2.png from 192.168.54.250:3000
21 2024-11-16 09:04:18.125073: Successfully download piece 18 belong to img2.png from 192.168.54.250:3000
22 2024-11-16 09:04:18.125073: Successfully download piece 16 belong to img2.png from 192.168.54.250:3000
23 2024-11-16 09:04:18.139306: Successfully download piece 19 belong to img2.png from 192.168.54.250:3000
24 2024-11-16 09:04:18.139306: Successfully download piece 20 belong to img2.png from 192.168.54.250:3000
25 2024-11-16 09:04:18.151897: Successfully download piece 21 belong to img2.png from 192.168.54.250:3000
26 2024-11-16 09:04:18.157894: Successfully download piece 23 belong to img2.png from 192.168.54.250:3000
27 2024-11-16 09:04:18.157894: Successfully download piece 22 belong to img2.png from 192.168.54.250:3000
```

Hình 23: File log ghi lại nội dung của quá trình download



Hình 24: File hình ảnh được tải thành công

Ta tiếp tục download một file có nhiều peer sở hữu hơn. Ta sẽ thực hiện download file có độ lớn khoảng 150MB, có 3 peer đang seeding là 192.168.54.250, 192.168.54.113 và 192.168.54.31. Tuy nhiên 192.168.54.31 không sở hữu các piece 137, 244, 248, 249, 250, 462, 463, 464. Kết quả download log như dưới đây.

```
2024-11-16 10:22:44.167560: Start downloading, meatafile: 4459aff372d3deda6aa87035b993dca173277e5a
2024-11-16 10:22:44.177312: Establish connection to peer S2XQEAI2F5-192.168.54.250:3000
2024-11-16 10:22:44.410089: Establish connection to peer 0I07KASV2R-192.168.54.113:3000
2024-11-16 10:22:45.650607: Establish connection to peer GYA6J8A2CB-192.168.54.31:3000
2024-11-16 10:22:45.935238: Successfully download piece 244 belong to glove.6B.50d.txt from 192.168.54.250:3000
2024-11-16 10:22:45.935238: Successfully download piece 250 belong to glove.6B.50d.txt from 192.168.54.250:3000
2024-11-16 10:22:45.935238: Successfully download piece 249 belong to glove.6B.50d.txt from 192.168.54.250:3000
2024-11-16 10:22:45.977705: Successfully download piece 462 belong to glove.6B.50d.txt from 192.168.54.250:3000
2024-11-16 10:22:46.009554: Successfully download piece 137 belong to glove.6B.50d.txt from 192.168.54.113:3000
2024-11-16 10:22:46.031890: Successfully download piece 248 belong to glove.6B.50d.txt from 192.168.54.113:3000
2024-11-16 10:22:46.037117: Successfully download piece 0 belong to img1.png from 192.168.54.250:3000
2024-11-16 10:22:46.041930: Successfully download piece 1 belong to glove.6B.50d.txt from 192.168.54.250:3000
2024-11-16 10:22:46.207593: Successfully download piece 463 belong to glove.6B.50d.txt from 192.168.54.113:3000
2024-11-16 10:22:46.245970: Successfully download piece 3 belong to glove.6B.50d.txt from 192.168.54.250:3000
2024-11-16 10:22:46.466259: Successfully download piece 464 belong to glove.6B.50d.txt from 192.168.54.113:3000
2024-11-16 10:22:46.494431: Successfully download piece 5 belong to glove.6B.50d.txt from 192.168.54.31:3000
2024-11-16 10:22:46.511098: Successfully download piece 6 belong to glove.6B.50d.txt from 192.168.54.250:3000
2024-11-16 10:22:46.526693: Successfully download piece 4 belong to glove.6B.50d.txt from 192.168.54.31:3000
2024-11-16 10:22:46.526693: Successfully download piece 2 belong to glove.6B.50d.txt from 192.168.54.31:3000
2024-11-16 10:22:46.574002: Successfully download piece 10 belong to glove.6B.50d.txt from 192.168.54.250:3000
2024-11-16 10:22:46.621088: Successfully download piece 13 belong to glove.6B.50d.txt from 192.168.54.250:3000
```

Hình 25: Những dòng đầu tiên của download log

Ta thấy vì chỉ có 2 peer sở hữu các mảnh 137, 244, ... như trên nên đây là các piece 'hiếm'. Theo chiến lược rarest-first, nó sẽ được download đầu tiên đúng như trong file log. Thứ tự download cũng không tăng dần mà có tính ngẫu nhiên do sự xáo trộn giữa các piece có cùng rarity và do xử lý song song.

Có thể thấy, client gửi yêu cầu và nhận piece đồng thời từ cả 3 peer.

```
2024-11-16 10:23:28.394390: Successfully download piece 654 belong to glove.6B.50d.txt from 192.168.54.250:3000
2024-11-16 10:23:28.394390: Successfully download piece 653 belong to glove.6B.50d.txt from 192.168.54.31:3000
2024-11-16 10:23:28.493215: Successfully download piece 652 belong to glove.6B.50d.txt from 192.168.54.113:3000
2024-11-16 10:23:28.493215: Successfully download piece 651 belong to glove.6B.50d.txt from 192.168.54.113:3000
```

Hình 26: Những piece cuối cùng được tải về vào lúc 10:23:28.

Tổng thời gian tải là khoảng 44 giây.

5.8 Chức năng upload

Chức năng upload trong suốt quá trình được ghi lại qua file log.

```

2024-11-16 17:55:40.421843: Handshake with ('127.0.0.1', 51530)
2024-11-16 17:55:40.421843: Send bitfield: 1111111 to ('127.0.0.1', 51530)
2024-11-16 17:55:40.433741: Receive request (id=6) from ('127.0.0.1', 51531) for piece 1 of 6eca34d6993d70f1aba79fc1875bdb2e5a80eef8
2024-11-16 17:55:40.435604: Send piece 1 of 6eca34d6993d70f1aba79fc1875bdb2e5a80eef8 to ('127.0.0.1', 51531)
2024-11-16 17:55:40.436235: Receive request (id=6) from ('127.0.0.1', 51532) for piece 0 of 6eca34d6993d70f1aba79fc1875bdb2e5a80eef8
2024-11-16 17:55:40.436235: Receive request (id=6) from ('127.0.0.1', 51533) for piece 2 of 6eca34d6993d70f1aba79fc1875bdb2e5a80eef8
2024-11-16 17:55:40.436235: Send piece 0 of 6eca34d6993d70f1aba79fc1875bdb2e5a80eef8 to ('127.0.0.1', 51532)
2024-11-16 17:55:40.437742: Receive request (id=6) from ('127.0.0.1', 51534) for piece 5 of 6eca34d6993d70f1aba79fc1875bdb2e5a80eef8
2024-11-16 17:55:40.438634: Receive request (id=6) from ('127.0.0.1', 51535) for piece 3 of 6eca34d6993d70f1aba79fc1875bdb2e5a80eef8
2024-11-16 17:55:40.437742: Send piece 2 of 6eca34d6993d70f1aba79fc1875bdb2e5a80eef8 to ('127.0.0.1', 51533)
2024-11-16 17:55:40.438634: Send piece 5 of 6eca34d6993d70f1aba79fc1875bdb2e5a80eef8 to ('127.0.0.1', 51534)
2024-11-16 17:55:40.438634: Send piece 3 of 6eca34d6993d70f1aba79fc1875bdb2e5a80eef8 to ('127.0.0.1', 51535)
2024-11-16 17:55:40.441227: Receive have message (id=4) from ('127.0.0.1', 51534) for piece 5
2024-11-16 17:55:40.441227: Receive have message (id=4) from ('127.0.0.1', 51532) for piece 0
2024-11-16 17:55:40.441227: Receive have message (id=4) from ('127.0.0.1', 51535) for piece 3
2024-11-16 17:55:40.441227: Receive have message (id=4) from ('127.0.0.1', 51531) for piece 1
2024-11-16 17:55:40.441227: Receive request (id=6) from ('127.0.0.1', 51536) for piece 4 of 6eca34d6993d70f1aba79fc1875bdb2e5a80eef8
2024-11-16 17:55:40.452229: Send piece 4 of 6eca34d6993d70f1aba79fc1875bdb2e5a80eef8 to ('127.0.0.1', 51536)
2024-11-16 17:55:40.452229: Receive request (id=6) from ('127.0.0.1', 51537) for piece 6 of 6eca34d6993d70f1aba79fc1875bdb2e5a80eef8
2024-11-16 17:55:40.452229: Send piece 6 of 6eca34d6993d70f1aba79fc1875bdb2e5a80eef8 to ('127.0.0.1', 51537)
2024-11-16 17:55:40.453742: Receive have message (id=4) from ('127.0.0.1', 51537) for piece 6
2024-11-16 17:55:40.453742: Receive have message (id=4) from ('127.0.0.1', 51533) for piece 2
2024-11-16 17:55:40.456216: Receive have message (id=4) from ('127.0.0.1', 51536) for piece 4

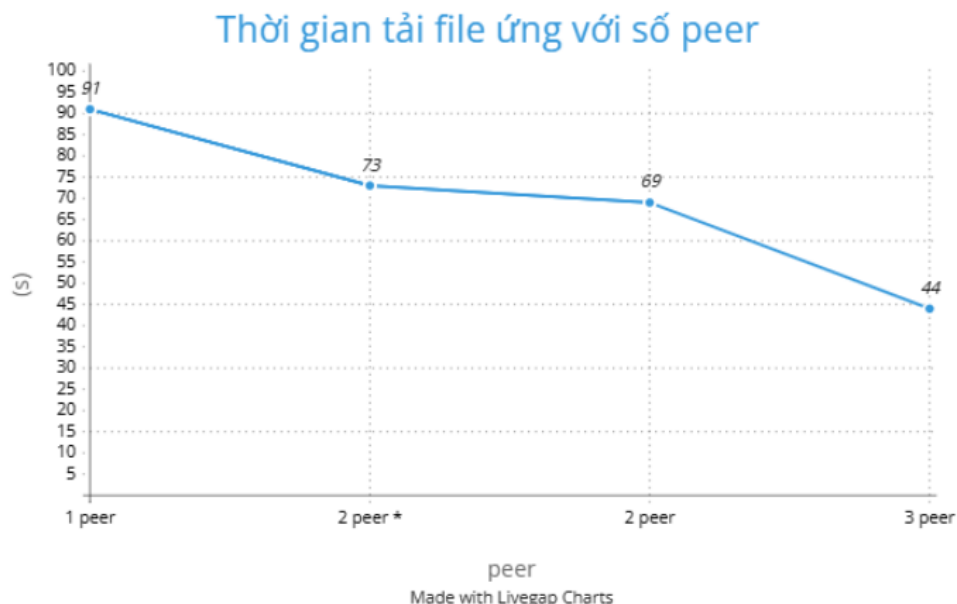
```

Hình 27: Ứng dụng có thể giải quyết nhiều yêu cầu gửi đến.

Ta thấy ứng dụng có thể thiết lập kết nối và trao đổi file. Ứng dụng có thể upload cho nhiều peer khác nhau một cách đồng thời.

6 Đánh giá hiệu suất

Ta thực hiện thí nghiệm tải file có kích thước 167MB. Đầu tiên, peer 1 sẽ tạo và publish file này. Một peer 2 khác sẽ tải trực tiếp file này về máy và ghi lại thời gian. Tương tự peer 3 sẽ tải file từ peer1 và peer2. Peer 4 sẽ tải file từ cả 3 peer. Kết quả đạt được như sau:



Hình 28: Kết quả so sánh thời gian tải. (2 peer * là trường hợp peer 2 chỉ có 100/167 MB của file)

7 Nhiệm vụ và đóng góp của các thành viên

- Phạm Ngọc Long: Thiết kế hệ thống, hiện thực hệ thống, kiểm thử.
- Nguyễn Quang Huy: Thiết kế hệ thống, viết báo cáo.
- Võ Phương Minh Nhật: Thiết kế hệ thống, kiểm thử.

8 Những điều đạt được và hướng mở rộng

Nhóm đã hiện thực được các chức năng chính của một ứng dụng chia sẻ file trong mạng P2P dựa trên bittorrent. Thực hiện thành công quá trình tải file đồng thời từ nhiều máy, qua đó chứng tỏ tốc độ và lợi thế của ứng dụng. Nhóm cũng đã hiện thực được một số tính năng mở rộng từ yêu cầu đề ra như:

- Lên chiến lược tải file từ nhiều peer: Được mô tả trong mục 4.3.
- Tải nhiều file đồng thời và upload nhiều file đồng thời. Được mô tả trong mục 4.3, 4.4.
- Thực hiện hash các piece trong khóa info để đảm bảo tính hợp lệ khi tải các piece từ nhiều nguồn.
- Tích hợp thêm khả năng tìm kiếm và tải các torrent file.

Tuy nhiên ứng dụng vẫn còn một số hạn chế như: server database không ổn định do database được triển khai online, xử lí đa luồng chưa tối ưu...

Ứng dụng có thể mở rộng thêm theo nhiều hướng như: Distributed hash table, Tracker scrape,...



Tài liệu tham khảo

- [1] BitTorrentSpecification. Được truy cập từ, <https://wiki.theory.org/BitTorrentSpecification>, ngày truy cập 11/11/2024.
- [2] Tracker scrape. Được truy cập từ, https://en.wikipedia.org/wiki/Tracker_scrape, ngày truy cập 11/11/2024.